Auditing the Blast Radius: Artifact Consistency in npm Supply-Chain Prioritization

Anonymous Authors
Research manuscript draft

***Abstract*—Dependency reach, shared maintainers, and public continuous integration workflows offer useful but incomplete views of software supply-chain exposure. Their combination into a predictive score requires internally consistent artifacts and a defensible outcome population. We audit an npm research repository containing 25,228 packages and two compressed metric artifacts. Independent recomputation reproduces all seven database metric fields, yet the companion CSV disagrees on workflow reach for 17,809 packages: only 20 have nonzero values in the CSV, compared with 17,809 in the database. At a one-percent review budget of 253 packages, their equal-weight score queues share only 65 entries. Large score ties further affect queue membership. Separately, the incident companion contains 47 package-event records and 45 retained pre-event source objects, but supplies no cutoff-valid control population. These findings establish artifact inconsistency and evaluation ineligibility, rather than predictive accuracy. We provide an offline audit, aggregate results, and an evidence-aware protocol separating structural exposure, workflow configuration, release authority, and future documented compromise. The contribution is a reproducible measurement study and benchmark specification; no learned model or attacker-return estimate is claimed.**

***Index Terms*—software supply chain, npm, dependency graphs, artifact consistency, defensive prioritization, temporal validity**

I. Introduction

Open-source package dependencies distribute both functionality and trust. Earlier npm research established that dependency concentration and maintainer sharing create systemic points of failure [1]. Metadata weak-link signals also support proactive review [2]. The practical question is therefore not whether package graphs matter, but what a particular graph-derived queue measures and whether the artifacts used to construct it support that interpretation.

The repository studied here was developed around a proposal to forecast high-impact supply-chain entry points using dependency, maintainer, and public GitHub data. Its implemented collector stores npm package metadata, deps.dev dependency edges, workflow-directory observations, and imported advisories. A deterministic metric program combines maintainer portfolio size with the count of downstream packages associated with public workflows. A graph neural network is discussed in project documents but is not trained or evaluated by the available implementation.

This setting exposes a prerequisite often hidden by model-centric evaluation: a plausible score can be computed from inconsistent exports. We find that the compressed CSV and SQLite archives contain the same package population and identical maintainer and dependency metrics, while their workflow-derived metrics differ substantially. Consequently, using a shared population count or a common repository revision as a snapshot identifier is insufficient. A researcher can reproduce a reported CSV statistic faithfully while drawing conclusions inconsistent with the database in the same checkout.

We study three questions. **RQ1:** Are the supplied metric artifacts mutually consistent and reproducible from their stored graph and workflow observations? **RQ2:** How much do artifact choice, score weighting, and boundary ties change a fixed-budget defensive review queue? **RQ3:** What empirical claims remain supportable given the historical incident evidence and missing comparison population?

The resulting contribution has three parts. First, we provide a read-only audit that independently reconstructs all database metrics and compares every package across artifacts. Second, we quantify cross-artifact disagreement, queue overlap, and tie sensitivity without treating structural scores as compromise probabilities. Third, we specify a future benchmark whose time, population, outcome, and evidence requirements are explicit. Its graph model remains a hypothesis to test after those requirements are satisfied.

Our central result is a measurement result: 17,809 of 25,228 packages disagree on workflow reach, although every recomputed database metric agrees with its stored value. The two equal-weight score queues retain only 65 of 253 entries in common. A separate inventory of the historical companion yields 47 event records across 31 family identifiers, 45 retained source objects, and no eligible controls. These results support an artifact-consistency paper and a benchmark design, but cannot establish incident forecasting performance.

II. Related Work and Novelty Boundary

A. Ecosystem Structure and Metadata Signals

Zimmermann et al. analyze npm dependencies, maintainers, and security issues, establishing the importance of concentrated trust [1]. Zahan et al. investigate metadata weaknesses, including install scripts, inactive maintenance, and expired maintainer email domains [2]. These studies preclude claiming maintainer degree or metadata hygiene as new concepts. Our contribution concerns the consistency and interpretation of artifacts used to operationalize those concepts.

B. Payload Detection and Incident Collections

Ohm et al. compile documented malicious packages and organize supply-chain attack techniques [3]. Amalfi com-

bines classifiers, source reproducibility checks, and clone detection [4]. Guo et al.'s updated empirical study compares malicious-package detectors on a unified benchmark [5]. These tasks evaluate released package content. A package classified as benign in a release corpus is not automatically a non-event control for whether its publisher is compromised in a later horizon. We do not transfer detection performance or labels into a prospective ranking task.

C. Public Workflows and Release Authority

Koishybayev et al. measure GitHub workflow security properties, while ARGUS analyzes code injection through staged taint analysis [6], [7]. Workflow-directory presence is therefore a much coarser observation than the existing security analyses. GitHub documentation provides concrete configuration semantics for permissions and privileged triggers [8]. Santos-Grueiro's release-authority measurement compares predecessor releases across public authority evidence and reports that its review triggers do not overlap exact malicious versions in the studied cohort [9]. We consequently treat authority change as a separate review surface, not an established malware predictor.

D. Historical Data and Temporal Forecasting

npm-follower archives package releases, including material subsequently removed from the live registry [10]. CHRONO-RESOLUTION provides dependency resolution at release points [11]. Historical graph construction alone is therefore not a sufficient novelty claim. More directly, SupplyTGN studies cutoff-safe temporal risk ranking in an OSV-seeded PyPI setting [12]. PatchRisk studies future transitive vulnerability exposure for npm and PyPI using temporal and package-disjoint evaluation [13]. The latter was published after the repository's August research notes and materially narrows the novelty of a broad "leakage-aware package forecasting" claim. Its target concerns future vulnerability advisories, rather than adjudicated publisher compromise.

The defensible distinction is the empirical cross-artifact audit and its consequences for fixed-budget queues, coupled with an explicitly scoped compromise-benchmark specification. We do not claim the first temporal package benchmark, a new graph architecture, or a completed comparison of structural, workflow, and authority queues. Table I separates related contributions by their actual outcomes. The literature screen was updated on 20 October 2026; it is a targeted novelty assessment, not an exhaustive systematic review.

III. Problem Definition and Evidence Model

A. Observed Graph and Legacy Metrics

Let P, M, and R denote observed packages, registry maintainer identities, and mapped repositories. A dependency edge $(u, v) \in E$ points from a dependent package to its dependency. The implemented graph projects all stored version pairs onto package names. Its reverse transitive dependent set is

$$\mathcal{D}(p) = \{q \in P \setminus \{p\} : q \rightsquigarrow p\}. \tag{1}$$

Let $M(p)$ be the observed maintainer set and $d(m)$ the number of packages associated with identity m. The implemented portfolio statistics are

$$W_{\max}(p) = \max_{m \in M(p)} d(m), \qquad W_{\Sigma}(p) = \sum_{m \in M(p)} d(m), \tag{2}$$

with zero for an empty maintainer set. W_{Σ} can count overlapping portfolios repeatedly; neither statistic demonstrates that an observed identity has usable publication authority at an earlier date.

For a package q, the collector-derived flag $h(q)$ is one when any stored version has a positive public workflow-directory observation. The implemented workflow reach is

$$B(p) = \sum_{q \in \mathcal{D}(p)} h(q). \tag{3}$$

This is a count of packages with a repository-level directory observation. It does not establish that a workflow installs p, uses the relevant release, executes successfully, or has privileged credentials. Several packages may map to the same repository. Thus B does not count distinct organizations, workflows, installations, or compromised systems.

The legacy score, stored under `attacker_roi`, is

$$S_{\alpha}(p) = \alpha \frac{W_{\max}(p)}{\max_q W_{\max}(q)} + (1 - \alpha) \frac{B(p)}{\max_q B(q)}. \tag{4}$$

Zero denominators are replaced with one. We evaluate the implementation default $\alpha = 0.5$ and a descriptive weight sweep. Neither score component observes attack cost or success probability, so the score has no established return-on-investment interpretation.

B. Separating Four Decision Targets

Structural exposure asks where code could flow within the observed graph. Workflow review asks which public configurations deserve inspection. Release-authority review asks whether a release path differs from its predecessor. Future documented compromise asks whether an eligible package experiences a specified incident after a cutoff. These targets require different evidence and denominators. A score useful for one target need not predict another.

In a future compromise study, define an at-risk population $\mathcal{P}_t$ before outcome inspection and an adjudicated event time T_p. For a horizon H,

$$Y_{p,t,H} = \mathbb{1}\{T_p \in (t, t + H]\}. \tag{5}$$

An unresolved outcome remains unknown rather than becoming zero. Even a reviewed zero means no documented qualifying event under the stated surveillance protocol, not proof of intrinsic benignness.

C. Evidence and Availability

Every proposed feature should retain its value, source, capture time, claimed validity interval, fidelity class, and missingness reason. A version published before t but queried after t is a retrospective source object, not automatically

TABLE I
POSITIONING AGAINST THE CLOSEST RESEARCH STREAMS. DIFFERENCES IDENTIFY SCOPE, NOT SUPERIORITY.

Work	Principal evidence	Evaluated target or artifact	Boundary for this study
Small World; Weak Links [1], [2]	Dependencies, maintainers, metadata	Concentration and metadata weaknesses	Structural descriptors already established
Amalfi; detector benchmark [4], [5]	Released package content	Malicious-package detection	Detection labels do not define future non-events
CI characterization; ARGUS [6], [7]	Workflow and action code	Configuration properties; injection paths	Directory presence is a weaker observation
On Good Authority [9]	Predecessor release-path evidence	Authority discontinuities	Authority change and compromise differ
npm-follower; CHRONO-RESOLUTION [10], [11]	Archived releases; release-point resolution	Historical data resources	Graph reconstruction alone is not new
SupplyTGN; PatchRisk [12], [13]	Temporal package/advisory evidence	Future vulnerability-risk ranking	Temporal forecasting is already studied
This study	Two local archives; incident-record inventory	Metric consistency, queue drift, feasibility	No fitted model or predictive benchmark result

an immutable historical snapshot. Historical authority and workflow claims require dated supporting records of their own. A SHA-256 digest establishes the identity of retained bytes; it does not establish that those bytes were public or unchanged at a past cutoff.

We distinguish measured audit findings, inventories of existing repository records, literature-supported premises, and proposed experiments. This prevents a label registry from becoming an evaluated prediction corpus merely because both are stored alongside graph data.

IV. AUDIT METHODOLOGY

A. Inputs and Scope

The audit uses the committed `data/metrics_summary.csv.zip` and `data/npm_ecosystem.db.zip`. Their SHA-256 digests begin `43e1845a1067` and `ab45439cf94c`, respectively; full digests and the repository revision are recorded in the supplied JSON manifest. The study date is 20 October 2026. It is not asserted to be a data-collection date.

The database has 25,228 packages, 22,340 package-version rows, 12,217 maintainers, 79,405 package–maintainer relations, 121,885 version-specific dependency rows, and 153,488 advisories. The initial seed mechanism uses a high-impact package list and dependency discovery. Accordingly, the population is a convenience sample produced by that collector, not an npm census or probability sample.

B. Independent Metric Reconstruction

The audit extracts the database into a temporary directory and opens it read-only. It verifies uniqueness and equality of package populations, derives portfolio sizes from stored maintainer relations, projects dependency edges onto package-name pairs, and traverses reverse edges with duplicate suppression. It excludes the starting package even in a cycle, applies the workflow flag as implemented, and recomputes all seven fields independently of the production metric program. Integer fields are compared exactly and database scores at absolute tolerance 10^{-12}.

The CSV stores scores to six decimal places. Cross-artifact score differences therefore use tolerance 5.00001×10^{-7}; we additionally compare the database score rounded to six decimals with the export. This distinguishes numeric serialization from substantive disagreement. The audit never installs a package, retrieves a tarball, or modifies either source archive.

C. Review Queues and Boundary Ties

For budget $K = \lceil 0.01|P| \rceil = 253$, let $Q_K^A(f)$ be the top-K package set under field f in artifact A. Scores descend; equal values use ascending Unicode code-point order of package names. This is explicit because locale-dependent ordering can change boundary membership. We report intersection and Jaccard overlap,

$$J(Q, Q') = \frac{|Q \cap Q'|}{|Q \cup Q'|}, \tag{6}$$

and repeat with descending package names. Neither comparison evaluates security effectiveness.

For each boundary, let a packages score strictly above the Kth score, b tie at that score, and $s = K - a$ slots remain. Under uniform tie breaking, an above-boundary package has inclusion probability one, a tied package s/b, and any other package zero. For independently broken ties in two queues,

$$\mathbb{E}[|Q \cap Q'|] = \sum_{p \in P} \pi_p \pi'_p. \tag{7}$$

This is a combinatorial sensitivity calculation, not a confidence interval. A deterministic name rule makes a queue reproducible but does not give otherwise tied packages an evidence-based priority.

D. Weight and Incident-Inventory Audits

We recompute Equation 4 separately within each archive at $\alpha \in \{0, 0.1, \ldots, 1\}$, retaining each archive's own maxima. No weight is selected using incident outcomes. Spearman correlations use average ranks for ties.

The separate temporal inventory reads the package-event snapshot CSV and direct-declaration CSV under

TABLE II
CROSS-ARTIFACT COMPARISON. POSITIVE COUNTS AND PACKAGE-LEVEL DISAGREEMENTS; SCORE DIFFERENCES ALLOW CSV ROUNDING.

Field	CSV > 0	DB > 0	Different
Maintainer portfolio (max)	22,147	22,147	0
Maintainer portfolio (sum)	22,147	22,147	0
Workflow reach	20	17,809	17,809
Direct dependents	20,331	20,331	0
Workflow direct dependents	15	15,017	15,014
Transitive dependents	20,328	20,328	0
Legacy score	22,147	24,502	17,809

`platform/research/`. It counts records and family identifiers, checks recorded source timestamps against cutoffs, and hashes those inputs. It does not repeat the upstream incident adjudication or retrieve the original registry responses. Its results characterize repository readiness, not independent verification of every incident claim.

V. EMPIRICAL RESULTS

A. RQ1: Metric and Artifact Consistency

All seven database metric fields reproduce with zero mismatches under the declared tolerances. The two archives contain identical sets of 25,228 package names. Their maintainer portfolio, direct-dependent, and transitive-dependent values agree for every package. The workflow-derived quantities do not (Table II).

The CSV has 20 nonzero workflow-reach entries (0.0793% of packages), while the database has 17,809 (70.5922%). Their maxima are two and 3,310, respectively. Workflow-bearing direct-dependent counts disagree for 15,014 packages; 15 CSV entries and 15,017 database entries are positive. The equal-weight score differs for the same 17,809 packages as workflow reach, even after accounting for six-decimal CSV serialization. This pattern localizes the disagreement to workflow-derived metrics in these artifacts; it does not establish when or why the divergence occurred.

The database contains 15,300 positive workflow observations across version rows, 6,223 checked negatives, and 817 unchecked rows. Positive observations cover 15,233 distinct packages. Therefore the sparse CSV workflow reach cannot be interpreted as evidence that workflow information is generally absent from the stored database. Figure 2 contrasts the artifact-level summaries.

The platform snapshot manifest assigns a June 2026 snapshot date, while every stored metric row carries `2026-07-14 06:39:51` as its calculation timestamp. These describe different events, and the repository does not supply enough capture history to establish a coherent historical collection time for all inputs. We use archive digests as identities and report this timing limitation explicitly. We neither overwrite the supplied exports nor silently select a shared historical date.

B. RQ2: Queue Drift at a Fixed Review Budget

At $K = 253$, the equal-weight score queues intersect in 65 packages, or 25.69% of each queue. Their Jaccard overlap is 0.1474. Thus 188 selected entries change when choosing one archive rather than the other. The workflow-reach queues share 20 packages and have Jaccard overlap 0.0412. By contrast, the dependency-reach queues match exactly (Table III).

TABLE III
CROSS-ARTIFACT TOP-253 QUEUE OVERLAP UNDER THE SPECIFIED NAME ORDERING. EXPECTED OVERLAP INDEPENDENTLY RANDOMIZES BOUNDARY TIES.

Queue	Shared	Jaccard	Expected shared
Equal-weight score	65	0.1474	55.14
Dependency reach	253	1.0000	253.00
Workflow reach	20	0.0412	22.15

Boundary ties qualify these exact set comparisons. In the CSV score queue, only 16 packages exceed the boundary value of 0.5; 967 tie there and compete for 237 remaining slots. The database queue has 250 above-boundary packages and six tied packages competing for three slots. Reversing the name ordering changes the shared score count from 65 to 53. Equation 7 gives an expected count of 55.14 under independent uniform tie breaking. These are alternative queue constructions, not repeated statistical samples.

For CSV workflow reach, all 20 positive entries fit within the budget and the remaining 233 slots come from 25,208 zeros. Hence most selected entries have no positive signal from that field. A deterministic top-K list can conceal this lack of resolution. Review interfaces should report score tiers and the number of tied slots rather than implying a meaningful strict order.

The weight sweep demonstrates that the disagreement persists beyond one coefficient (Figure 3). For $\alpha = 0$ through 0.3, shared membership is 20/253; at 0.5 it is 65/253; and at 0.9 it is 95/253. At $\alpha = 1$, the queues match because the score depends only on the identical maintainer component. This is a consistency endpoint, not evidence that a maintainer-only queue is operationally superior. We do not optimize weights against incident labels.

Within each archive, the legacy score also differs from dependency reach. Their top-253 intersections are 16 for the CSV and 63 for the database. Spearman correlations are 0.0825 and 0.2521, respectively. These describe distinct rankings and the influence of artifact choice. They do not show that either score predicts compromise, or that agreement with centrality is a desirable outcome.

C. Concentration and Graph-Semantics Checks

The top 123 maintainers, representing the rounded-up one-percent maintainer budget, account for 18,027 of 79,405 package–maintainer relations (22.70%). The largest portfolio contains 967 packages. These measured concentrations motivate attention to shared identities, while leaving actual publication authority and compromise likelihood unresolved.

The 121,885 stored dependency rows reduce to 88,113 distinct package-name pairs. The collector stores the destination node's `DIRECT`/`INDIRECT` status from a deps.dev root query

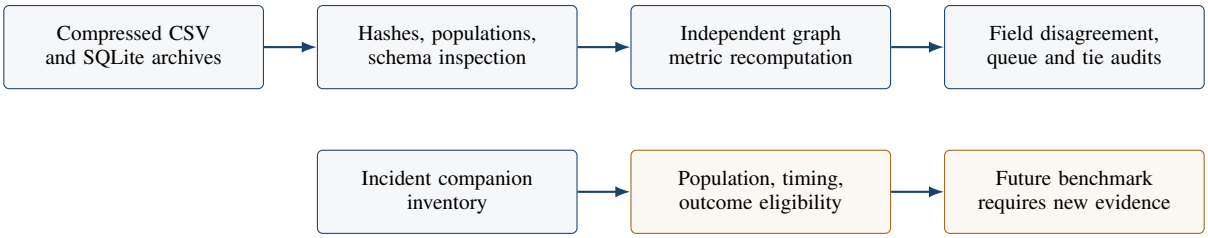


Fig. 1. Completed audit (upper path) and benchmark feasibility assessment (lower path). The lower path does not produce a trained model. Source hashes and fidelity labels accompany aggregate outputs.

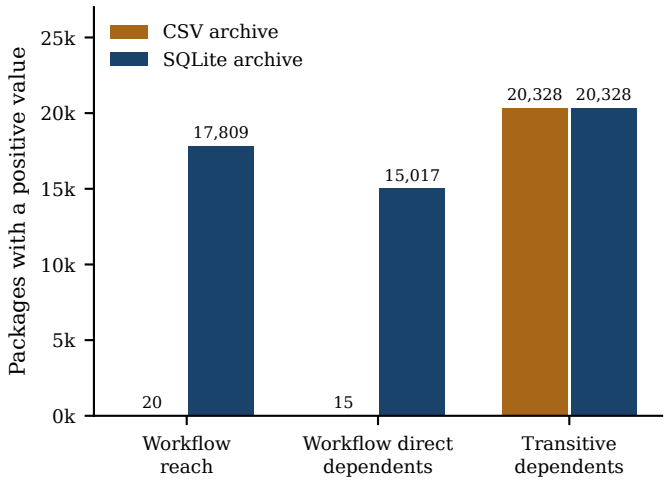


Fig. 2. Positive metric counts from the two archives. The same package population and unchanged dependency metrics coexist with large workflow-derived disagreement.

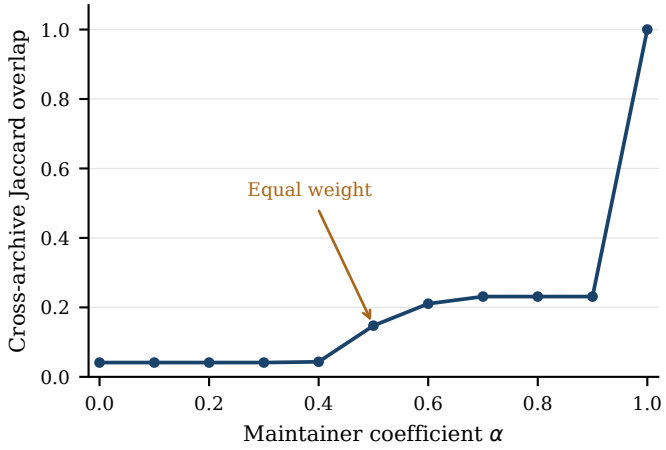


Fig. 3. Cross-artifact queue overlap as the maintainer coefficient increases. Each artifact uses its own normalization maxima. The matching endpoint removes the inconsistent workflow component.

in each row's `relation` field. Official API semantics define that status relative to the queried root, whereas graph edges identify the node declaring a dependency [14]. Consequently, filtering stored rows to `relation=DIRECT` would not generally recover all direct edges of the union graph. Our audit follows the implemented projection over every stored edge; it makes no claim of historical installed-tree fidelity. More broadly, projecting multiple versions can join paths that no single installation realized.

D. RQ3: Temporal Evidence Supports an Inventory, Not a Fit

The current snapshot companion contains 47 package-event records across 31 family identifiers, of which 45 have source objects tagged `registry_reconstructed_exact`. Those retained records span 30 family identifiers and contribute 258 runtime dependency declarations. Their recorded source publication timestamps precede their cutoffs. This check establishes consistency among the recorded fields; it does not independently authenticate their source responses or reconstruct historical workflow and authority states.

The repository supplies no cutoff-valid non-event comparison population for those events. Its two non-retained source records remain label-only exceptions. Family identifiers are not asserted to be independent attacks: the existing research notes identify campaign relatedness, and protestware, malicious new publications, and publisher compromise also differ in target meaning. Pooling these categories without an explicit estimand would obscure what a model learned.

The larger advisory table does not remove this limitation. Only 164 advisory package names overlap the metric population, 153,422 rows carry the broad classification `compromise`, and its schema has no affected-version relation. These imported records can support candidate discovery, but their volume does not establish version-specific outcomes or a defensible non-event population.

We therefore fit no model and report no PR-AUC, Recall@K, calibration curve, or predictive confidence interval. A positive-only convenience sample cannot determine a population false-positive rate or calibrated event probability. The measured contribution remains the descriptive audit, while the next section specifies the evidence needed to evaluate the original prediction question.

VI. Design of a Future Evidence-Aware Benchmark

This section is a proposed protocol. It is not an implemented or evaluated learning system. Figure 1 separates its evidence gate from the completed offline audit.

TABLE IV
REQUIRED EVIDENCE DISTINCTIONS IN THE PROPOSED BENCHMARK. CURRENT AUDIT FINDINGS DO NOT FILL MISSING HISTORICAL FIELDS.

Evidence	Permitted interpretation
Stored package projection	Reach in the observed union graph
Retained source declaration	Declared range in an identified version object
Cutoff SemVer selection	An explicitly bounded resolution approximation
Historical lockfile	Resolution recorded for a particular project and context
Dated workflow YAML	Observable configuration at a documented revision
Authority evidence	Specific release-path observation, with scope and predecessor

A. Freeze the Population and Outcome First

Before inspecting future incidents, freeze a registry-derived package population at cutoff t, inclusion criteria, observation unit, and follow-up horizon. The primary outcome should be documented compromise of an established package's publication path. Malicious packages created from inception, ordinary vulnerabilities, downstream exposure, and maintainer protestware should be separately stratified. A 30-day primary horizon and 90-day sensitivity horizon are reasonable design choices, but require registration before scoring.

For each incident, retain affected package versions, event and disclosure times, source URLs, evidence grade, campaign linkage, adjudicator decisions, and uncertainty intervals when exact timing is unavailable. Controls should be eligible packages observed over the same horizon under a documented surveillance protocol. Unknown outcomes must remain unknown. Sampling controls after seeing cases requires explicit sampling probabilities and a correspondingly restricted interpretation; case–control prevalence does not automatically support population calibration.

B. Freeze Features and Their Availability

For each package–cutoff pair, preserve source objects and retrieval provenance independently from outcome evidence. The feature record should include dependency-resolution mode, repository mapping, maintainer versus publisher semantics, missingness, and source-specific timing. Table IV gives the minimum distinctions. A version-time map can identify an eligible source release but cannot by itself prove historical workflow content, installation state, or authority.

Historical dependency resources should be reused where their dates and semantics match the target population [11]. npm documentation distinguishes a lockfile's exact generated tree from a version constraint [15]. None of these automatically identifies which private systems executed a release. Workflow features should encode observable permissions, relevant trigger/checkout combinations, action immutability, and missingness. Authority features should preserve predecessor evidence and mark unavailable comparisons as unknown [9].

C. Compare Transparent Models Before Graph Learning

The baseline suite should include a seeded random queue, package age and release activity, direct and transitive reach, maintainer portfolio descriptors, a weak-link metadata baseline, and regularized logistic regression. Popularity requires an actually dated source; current download counts cannot be backdated. Gradient-boosted trees provide a further tabular comparator when the outcome sample permits fitting.

Only then should a relation-aware graph model be tested against matched features and observation windows. A possible heterogeneous graph links package versions, maintainer identities, and repositories using dependency, maintenance, and mapping relations. Temporal message passing must respect the same cutoff rules as every baseline. Heterogeneity motivates a modeling option, not an assumption that graph attention is necessary. Attention coefficients should not be treated as causal explanations.

The main ablations should add maintainer, workflow, and authority evidence separately to a dependency-only model, plus missingness-only and metadata-only controls. A model's benefit must persist beyond simple release activity and observation coverage. Separate queues should also be compared at the same review budget before learning a fused score. A useful outcome may be that an interpretable queue or late evidence gate works as well as a complex model.

D. Evaluate Decision Value Without Leakage

Use chronological training, validation, and untouched test windows. Keep related campaign records within one evaluation unit, and conduct a package-disjoint sensitivity analysis to test identity memorization. Repeated cutoffs and overlapping horizons require purging or other explicit separation so that the same outcome is not shared across partitions. All preprocessing, coefficient tuning, thresholds, and calibration must use training or validation data only.

For a fully adjudicated test population, report Precision@K, Recall@K, ranking utility at several registered budgets, and PR-AUC relative to the observed base rate. State how unknown outcomes affect the denominator. Probability calibration requires representative prevalence or valid sampling correction; otherwise restrict the claim to ranking within the sampled population. Report uncertainty by resampling independent campaign groups or time blocks, explain the effective number of units, and avoid package-row bootstrap intervals that treat one campaign as many independent attacks.

SupplyTGN and PatchRisk already establish relevant temporal evaluation precedents [12], [13]. The additional research question here is whether separately timed workflow and authority evidence helps adjudicated compromise review after artifact consistency and population eligibility are established. Success would be incremental decision value under these controls; a null result would still clarify the limits of public metadata.

E. Executable Release Gate

A future predictive run should require a manifest binding source hashes, observation population, feature cutoffs, outcome protocol, campaign groups, split assignments, ranking configuration, and evaluation code. Reject missing historical evidence rather than substituting current values. A publication artifact should include all eligible predictions and exclusions, even when results are null. This paper's audit demonstrates the prerequisite consistency checks; it does not claim that the full predictive gate has already been implemented.

VII. Threats to Validity and Research Ethics

A. Construct and Internal Validity

Structural reach is conditional on the observed graph. Package projection removes version constraints; public workflow-directory observations do not establish execution, permissions, or installations. Maintainer usernames need not correspond one-to-one with current publishers or organizations. The legacy score's normalization is sample-dependent, and CSV rounding can enlarge score ties. These limitations apply even when recomputation succeeds.

Our audit localizes cross-artifact differences but does not reconstruct their cause. Differences could arise from workflow collection timing, export timing, or another undocumented process. We do not diagnose a specific fault without capture records. Matching database recomputation validates consistency with the stored inputs and formula, not correctness or completeness of the original collection.

The queue comparison uses a registered-in-code one-percent budget and an explicit name rule. Other budgets, normalization schemes, and tie policies can yield different values. The reported weight sweep is descriptive rather than a robustness guarantee. Deterministic totals over a finite artifact do not require sampling confidence intervals; they cannot be generalized to the full npm ecosystem using the same rationale.

B. External and Temporal Validity

The collector's high-impact seeding and recursive discovery overrepresent some parts of npm. Missing repositories and unchecked workflow states are not evidence of low risk. Several package observations can share a repository or maintainer, and related campaigns further reduce independent units. No ecosystem-wide compromise rate follows from these data.

The temporal companion is inventoried from existing CSVs. We do not independently re-adjudicate all 47 incident records, verify retained raw response bytes, or reproduce historical workflow snapshots. Pre-event source selection from a later registry response does not guarantee historical immutability. Source deletion and availability may depend on maliciousness; surviving records can therefore induce selection bias. npm-follower motivates retention at publication time, but its reported coverage is not assumed to cover our incident set [10].

The novelty screen covers selected primary papers and documentation available by 20 October 2026. It is not a systematic literature review and cannot establish an exhaustive absence of similar studies. Recent forecasting and authority-measurement work is explicitly acknowledged rather than excluded through a narrow terminology choice.

C. Data Handling and Reproducibility

The audit works offline on supplied archives, opens SQLite read-only, and publishes aggregate output rather than new lists labeling maintainers or packages as malicious. It neither installs suspicious packages nor collects secrets. The source repository retains its original artifacts; manuscript-specific results record their full hashes. The paper package includes source, figures, generated numeric tables, and rebuild instructions. Raw databases are referenced rather than duplicated in the writing package.

Author identities and affiliations are intentionally left for the actual contributors to supply. The document is a research draft, with a measured audit and a proposed benchmark; it does not assert peer review, acceptance, funding, institutional approval, or completed predictive validation.

VIII. Conclusion

A supply-chain prioritization score is meaningful only within an identified evidence state. In this repository, independent reconstruction reproduces the database's seven metric fields, while the paired CSV disagrees on workflow reach and score for 17,809 packages. At a 253-package review budget, the score queues share 65 entries under a stated tie rule. The same underlying package population therefore does not imply the same operational review queue.

The historical companion provides a useful incident-record starting point, but its 47 package-event records and 45 retained source objects lack a cutoff-valid control population. Predictive discrimination and calibration remain unevaluated. The next empirical step is to freeze a defensible population and historical evidence, then compare transparent baselines and separate review surfaces before fitting a graph model. The present contribution is an auditable account of artifact consistency, queue sensitivity, and the evidence required to make that next study credible.

References

[1] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *28th USENIX Security Symposium*. USENIX Association, 2019, pp. 995–1010. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

[2] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" arXiv:2112.10165v2, 2022, ICSE SEIP 2022; DOI: 10.1145/3510457.3513044. [Online]. Available: https://arxiv.org/abs/2112.10165v2

[3] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," arXiv:2005.09535, 2020, DIMVA 2020 preprint. [Online]. Available: https://arxiv.org/abs/2005.09535

[4] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," arXiv:2202.13953, 2022, ICSE 2022; DOI: 10.1145/3510003.3510104. [Online]. Available: https://arxiv.org/abs/2202.13953

[5] W. Guo, Z. Chen, Z. Xu *et al.*, "How effective are NPM malicious package detectors? a large-scale empirical study," arXiv:2603.27549v2, 2026, revised August 5, 2026. [Online]. Available: https://arxiv.org/abs/2603.27549v2

[6] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry, "Characterizing the security of GitHub CI workflows," in *31st USENIX Security Symposium*. USENIX Association, 2022, pp. 2747–2763. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

[7] S. Muralee, I. Koishybayev, A. Nahapetyan, G. Tystahl, B. Reaves, A. Bianchi, W. Enck, A. Kapravelos, and A. Machiry, "ARGUS: A framework for staged static taint analysis of GitHub workflows and actions," in *32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 6983–7000. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/muralee

[8] GitHub, "Secure use reference," GitHub Actions documentation, accessed October 20, 2026. [Online]. Available: https://docs.github.com/en/actions/reference/security/secure-use

[9] I. Santos-Grueiro, "On good authority: Release-authority measurement for registry-mediated package ecosystems," arXiv:2606.22593v2, 2026, preprint. [Online]. Available: https://arxiv.org/abs/2606.22593v2

[10] D. Pinckney, F. Cassano, A. Guha, and J. Bell, "npm-follower: A complete dataset tracking the NPM ecosystem," arXiv:2308.12545, 2023. [Online]. Available: https://arxiv.org/abs/2308.12545

[11] I. Rahman, J. Marley, R. Paramitha, and L. Williams, "CHRONO-RESOLUTION: A dependency resolution dataset at release points for npm, PyPI, and crates.io packages," arXiv:2607.15315, 2026, ICSME 2026 Tool Demonstration and Data Showcase Track. [Online]. Available: https://arxiv.org/abs/2607.15315

[12] T. P. Tran Truong, S. Ha, and X.-B. Le, "SupplyTGN: A reproducible leakage-safe benchmark for temporal software supply-chain risk," SSRN preprint 7194447, 2026, posted July 28, 2026. [Online]. Available: https://ssrn.com/abstract=7194447

[13] A. A. Shafin and K. M. Ahmed, "PatchRisk: Forecasting future vulnerability exposure in open-source dependency networks," arXiv:2609.13719, 2026, WI-IAT 2026 preprint. [Online]. Available: https://arxiv.org/abs/2609.13719

[14] Open Source Insights, "deps.dev API v3: GetDependencies," Official API documentation, accessed October 20, 2026. [Online]. Available: https://docs.deps.dev/api/v3/

[15] npm, "package-lock.json," npm CLI v11 documentation, accessed October 20, 2026. [Online]. Available: https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json/